

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ
THUẬT TP.HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN TÍ TUỆ NHÂN TẠO
ĐỀ TÀI: THIẾT KẾ VÀ XÂY DỰNG CHƯƠNG
TRÌNH TRÒ CHƠI SOKOBAN**

Giảng viên hướng dẫn: PGS.TS Hoàng Văn Dũng

Nhóm sinh viên thực hiện:

Đặng Gia Huy	22110146
Nguyễn Quốc Thịnh	24110340
Lê Lâm Lực	22110374

Thành phố Hồ Chí Minh, tháng 06 năm 2026

MỤC LỤC

I. PHẦN MỞ ĐẦU	1
1. Lý do chọn đề tài và mục tiêu	1
2. Phạm vi nghiên cứu.....	1
II. TỔNG QUAN BÀI TOÁN & MÔ HÌNH HÓA	1
1. Luật chơi	1
2. Mô hình hóa bài toán.	1
III. KIẾN TRÚC HỆ THỐNG	2
1. Kiến trúc tổng quát.....	2
2. Kiến trúc đa luồng bất đồng bộ	2
3. Cấu trúc dữ liệu của từng thuật toán	3
4. Cấu trúc dữ liệu và giải thuật hỗ trợ giao diện & hoạt ảnh.....	3
IV. CÁC THUẬT TOÁN TÌM KIẾM	5
1. Nhóm thuật toán tìm kiếm mù (Uninformed Search)	5
2. Nhóm thuật toán tìm kiếm có thông tin (Informed Search)	5
V. CÁC KỸ THUẬT TỐI ƯU CỐT LỖI.....	7
1. Hashing Trạng thái với Zobrist Keys (64-bit)	7
2. Gom nhóm di chuyển (Macro-moves).....	7
3. State-aware Heuristic (Bitmask Dynamic Programming)	7
4. Advanced Deadlock Detection	7
VI. KỊCH BẢN KIỂM THỬ VÀ ĐÁNH GIÁ	7
1. Nhóm kịch bản cơ sở (bản đồ nhỏ, cấu trúc đơn giản)	8
2. Nhóm kịch bản bế tắc	8
3. Nhóm kịch bản phân nhánh cao (bản đồ trung bình, nhiều cụm thủng).....	8
VII. ĐỊNH HƯỚNG PHÁT TRIỂN DỰ ÁN	8

I. PHẦN MỞ ĐẦU

1. Lý do chọn đề tài và mục tiêu

Sokoban là trò chơi giải đố đẩy thùng phù hợp để nghiên cứu và thực hiện trong lĩnh vực trí tuệ nhân tạo. Số lượng trạng thái có thể phát sinh trong quá trình chơi rất lớn, biến việc tìm lời giải tối ưu thành một bài toán tìm kiếm có độ phức tạp cao và là môi trường lý tưởng để áp dụng thuật toán. Nhóm quyết định lựa chọn đề tài nhằm xây dựng một hệ thống trò chơi hoàn chỉnh tích hợp AI.

Hệ thống đánh giá hiệu năng các thuật toán dựa trên các tiêu chí cụ thể: thời gian thực thi, số lượng trạng thái mở rộng, độ dài lời giải tìm được.

2. Phạm vi nghiên cứu

Dự án cài đặt bằng ngôn ngữ Python, áp dụng thư viện Pygame để xử lý giao diện đồ họa. Luồng xử lý thuật toán và giao diện được tách biệt. Các thuật toán AI cài đặt bao gồm: BFS, DFS, UCS, Greedy Search, A*, Weighted A*, và Beam Search.

II. TỔNG QUAN BÀI TOÁN & MÔ HÌNH HÓA

1. Luật chơi

Trò chơi diễn ra trên lưới 2D. Người chơi di chuyển theo bốn hướng (lên, xuống, trái, phải) để đẩy các thùng đến vị trí đích. Các quy tắc vật lý áp dụng:

- Chỉ được đẩy một thùng mỗi lần.
- Không được kéo thùng.
- Không đẩy thùng xuyên qua tường chắn hoặc thùng khác.
- Trò chơi kết thúc khi tất cả thùng được đưa đến vị trí đích.

2. Mô hình hóa bài toán.

- Bản đồ:** Biểu diễn dưới dạng ma trận hai chiều, trong đó mỗi ô trên lưới thể hiện một đối tượng cụ thể:

Ký tự	Ý nghĩa
#	Tường
@	Người chơi
\$	Thùng
.	Vị trí đích
*	Thùng đặt trên đích
+	Người chơi đứng trên đích
(Space)	Ô trống

- Không gian trạng thái:** Một trạng thái được xác định bởi vị trí hiện tại của người chơi và tập hợp vị trí của các thùng.
- Trạng thái đích:** Đạt được khi tập hợp vị trí thùng trùng khớp hoàn toàn với tập hợp vị trí đích.
- Thách thức:** Số lượng trạng thái khổng lồ, tồn tại nhiều trạng thái bế tắc (deadlock) khiến bài toán không thể giải được.

III. KIẾN TRÚC HỆ THỐNG

1. Kiến trúc tổng quát

Hệ thống trò chơi Sokoban AI Solver được xây dựng dựa trên nguyên lý tách biệt hoàn toàn giữa ba phân lớp cốt lõi: logic trò chơi, giao diện hiển thị và công cụ giải AI. Mô hình kiến trúc này đảm bảo tính độc lập cao, giúp hệ thống dễ dàng bảo trì, tối ưu hóa và kiểm thử các thuật toán tìm kiếm mà không làm ảnh hưởng đến cấu trúc hiển thị đồ họa.

a. Các thành phần chính

- **Lớp mô hình hóa dữ liệu:** Phân tích dữ liệu bản đồ đầu vào từ file văn bản ký tự thô (.txt), thiết lập hệ thống ma trận tường chắn, ô đích cố định, đồng thời khởi tạo bảng khóa Zobrist ngẫu nhiên 64-bit phục vụ mã hóa trạng thái sinh trắc hình học.
- **Lớp quản lý phiên chơi :** Lưu trữ cấu hình tọa độ hiện hành của nhân vật và hệ thống thùng. Phân lớp này chịu trách nhiệm kiểm tra tính hợp lệ của các bước dịch chuyển vật lý và duy trì ngăn xếp lịch sử để thực thi chức năng quay lui phục hồi.
- **Lớp điều phối và giao diện:** Quản lý vòng lặp ứng dụng đồng bộ ở tần số 60 FPS, tiếp nhận các sự kiện từ chuột, bàn phím và chịu trách nhiệm vẽ toàn bộ tài nguyên đồ họa lên màn hình hiển thị.
- **Lớp công cụ giải AI:** Chứa mã nguồn cài đặt của 7 thuật toán tìm kiếm không gian cùng các module tiền xử lý khoảng cách, phát hiện bế tắc và tính toán giá trị Heuristic.

b. Quy trình hoạt động tổng quát

1. **Khởi tải và tiền xử lý:** Ứng dụng đọc file map .txt, phân tích các ký tự quy ước để trích xuất tọa độ tường, người chơi và các ô đích. Hệ thống thực hiện một pha tìm kiếm ngược phát xuất từ toàn bộ các ô đích để tính toán và lưu sẵn bản đồ khoảng cách thực tế, đồng thời xác định các ô chết hình học.
2. **Vòng lặp tương tác:** Hệ thống chạy vòng lặp cập nhật liên tục. Người chơi có thể tự điều khiển nhân vật bằng các phím mũi tên hoặc phím W/A/S/D để thay đổi trạng thái.
3. **Kích hoạt và xử lý AI:** Khi người dùng chọn một thuật toán và nhấn phím space hoặc nút RUN AI, hệ thống sẽ thiết lập lại trạng thái ban đầu của màn chơi, khóa các tương tác thủ công và khởi tạo một luồng xử lý nền độc lập cho AI.
4. **Trả kết quả:** Sau khi tìm ra đường đi, hệ thống trả về một đối tượng kết quả SolveResult chứa danh sách các hành động di chuyển. Luồng chính tiếp nhận chuỗi hành động này qua hàng đợi, kích hoạt hoạt ảnh dịch chuyển tự động phối hợp với thuật toán Hungarian để kết xuất hình ảnh. Các thông số đo lường hiệu năng lập tức được hiển thị trên bảng Telemetry.

2. Kiến trúc đa luồng bất đồng bộ

Để ngăn chặn hoàn toàn hiện tượng đống băng hoặc treo giao diện người dùng khi các thuật toán AI thực hiện duyệt hàng triệu trạng thái trên các bản đồ Sokoban phức tạp, hệ thống áp dụng kiến trúc đa luồng song song:

- **Luồng chính:** Chịu trách nhiệm vận hành đồ họa Pygame, lắng nghe sự kiện từ người dùng, thực hiện hoạt ảnh mượt mà cho nhân vật và tính toán tốc độ khung hình hiển thị.

- **Luồng xử lý phụ:** Được khởi chạy động thông qua thư viện `threading.Thread` mỗi khi có lệnh yêu cầu tìm kiếm lời giải. Luồng này hoạt động hoàn toàn độc lập ở nền sau, tập trung toàn bộ tài nguyên CPU để tính toán mở rộng các trạng thái tìm kiếm.

Cơ chế giao tiếp và đồng bộ luồng:

- **Truyền kết quả qua `queue.Queue`:** Luồng phụ đóng gói `SolveResult` rồi đẩy vào `result_queue`; luồng chính nhận bằng `result_queue.get_nowait()` ở mỗi vòng lặp.
- **Kiểm soát hủy lệnh bằng `threading.Event`:** Khi người dùng hủy hoặc đổi map, luồng chính gọi `stop_event.set()`. Luồng phụ kiểm tra `stop_event.is_set()` sau mỗi 1024 nút để thoát sớm.
- **Cập nhật tiến độ thời gian thực:** Hai luồng chia sẻ `progress_state: List[int]`; luồng phụ cập nhật số nút đã mở rộng sau mỗi 1024 nút, còn luồng chính đọc để hiển thị tiến độ trên dashboard.

3. Cấu trúc dữ liệu của từng thuật toán

Mỗi thuật toán trong hệ thống đòi hỏi các cấu trúc dữ liệu đặc thù để quản lý tập biên tìm kiếm và tập trạng thái đã duyệt nhằm tối ưu hóa chi phí thời gian và bộ nhớ:

- **BFS:** Sử dụng `collections.deque` làm hàng đợi hai đầu (FIFO) cho tập biên. Tập đã duyệt là một `HashSet Set[int]` lưu trữ mã băm Zobrist 64-bit của trạng thái. Dữ liệu lưu vết cấu hình qua bảng băm `ParentMap = Dict[StateKey, Tuple[StateKey, PathSegment]]`.
- **DFS:** Sử dụng cấu trúc mảng `List[]` vận hành như ngăn xếp bằng lệnh `.append()` và `.pop()`. Tập đã duyệt tương tự BFS.
- **UCS, Greedy, A*, Weighted A*, Beam Search:**
 - **Tập biên:** Dùng mảng `List[]` kết hợp với thư viện `heapq` để tạo thành hàng đợi ưu tiên.
 - **Tập đã duyệt:** Dùng cấu trúc từ điển để ánh xạ từ mã định danh trạng thái sang giá trị chi phí thực tế $g(n)$ tối ưu nhất hiện có, cho phép loại bỏ sớm các nhánh đi vòng vèo.

4. Cấu trúc dữ liệu và giải thuật hỗ trợ giao diện & hoạt ảnh

Bên cạnh các thuật toán tìm kiếm cốt lõi, lớp giao diện của hệ thống cũng tích hợp các cấu trúc dữ liệu và giải thuật nâng cao để đảm bảo hiệu năng hiển thị và tối ưu hóa trải nghiệm người dùng:

a. Ứng dụng giải thuật Hungarian xử lý hoạt ảnh di chuyển của thùng

Trong Sokoban, các thùng không có định danh cố định nên thứ tự tọa độ có thể thay đổi sau mỗi trạng thái mới (do `sorted()`). Nếu vẽ trực tiếp, hình ảnh thùng sẽ bị nhảy vị trí, gây giật lag thị giác. Để khắc phục điều này, hệ thống áp dụng thuật toán Hungarian thông qua hàm `linear_sum_assignment` của thư viện `SciPy`:

- **Cấu trúc dữ liệu:** Hệ thống tạo ma trận chi phí `NDArray[np.float64]` kích thước $N \times N$, trong đó mỗi ô (i, j) lưu khoảng cách Euclidean giữa vị trí hiển thị hiện tại của thùng i (`self.anim_boxes[i]`) và vị trí đích j (`target_boxes[j]`):

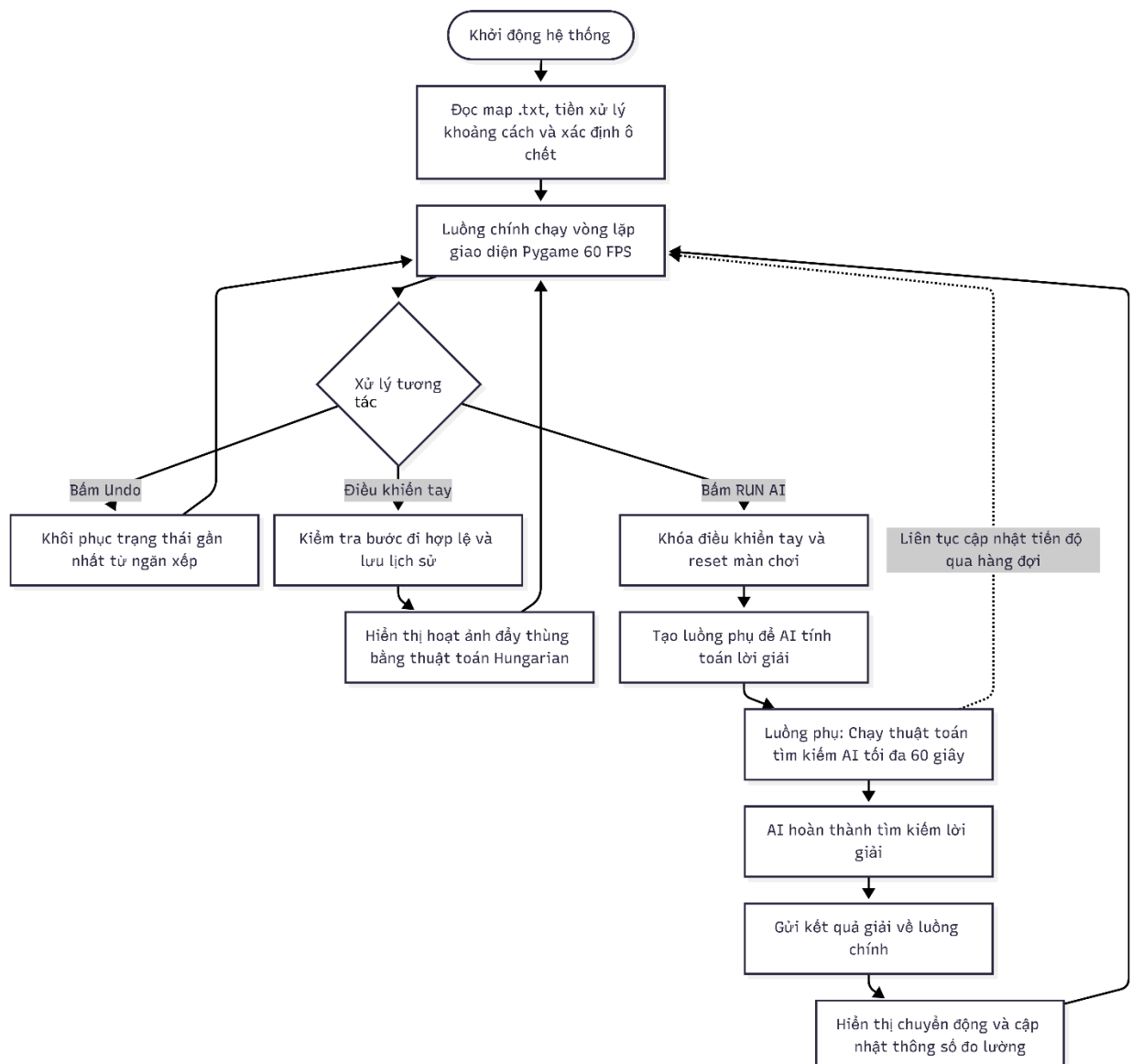
$$\text{cost_matrix}[i, j] = \sqrt{\left(r_{\text{anim_box}_i} - r_{\text{target_box}_j}\right)^2 + \left(c_{\text{anim_box}_i} - c_{\text{target_box}_j}\right)^2}$$

- **Logic giải thuật:** Hàm `linear_sum_assignment` tìm phương án ghép cặp thùng–đích có tổng khoảng cách nhỏ nhất thông qua hai mảng chỉ số `row_ind` và `col_ind`. Từ đó, hệ thống nội suy để cập nhật chuyển động mượt cho từng thùng đến đúng vị trí logic theo thời gian thực dựa trên tham số dt.

b. Lịch sử lưu vết phục vụ tính năng undo

Để hỗ trợ người chơi hoặc bot AI có thể lùi lại các bước đi sai lầm một cách nhanh chóng, hệ thống duy trì cấu trúc dữ liệu lịch sử trong lớp GameSession:

- **Cấu trúc dữ liệu:** Một danh sách tuần tự kiểu `List[State]` được vận hành như một cấu trúc ngăn xếp chuyên biệt.
- **Quy trình xử lý:** Mỗi khi thực hiện nước đi hợp lệ qua hàm `move()`, trạng thái `State` trước khi thay đổi (gồm vị trí người chơi, vị trí các thùng và mã băm Zobrist) sẽ được lưu vào ngăn xếp bằng `.append(self.state)`. Khi người dùng nhấn phím `U` hoặc nút `UNDO`, hàm `undo()` sẽ kiểm tra ngăn xếp và dùng `.pop()` để khôi phục trạng thái gần nhất. Cả hai thao tác đều có độ phức tạp thời gian $O(1)$, gần như không tốn tài nguyên hệ thống.



IV. CÁC THUẬT TOÁN TÌM KIẾM

1. Nhóm thuật toán tìm kiếm mù (Uninformed Search)

a. Tìm kiếm theo chiều rộng (BFS - Breadth-First Search)

- **Mô tả:** Thuật toán duyệt không gian trạng thái theo từng tầng chiều rộng, mở rộng tất cả các nút ở độ sâu hiện tại trước khi tiến xuống tầng sâu hơn thông qua cấu trúc dữ liệu hàng đợi (Queue - FIFO). Trong mã nguồn, BFS kết hợp cơ chế macro-moves để chỉ duyệt qua các trạng thái sinh ra từ hành động đẩy thùng hợp lệ nhằm giảm thiểu hệ số phân nhánh.
- **Điểm mạnh:** Đảm bảo tính đầy đủ cao, chắc chắn tìm thấy lời giải nếu tồn tại. Cam kết tìm ra chuỗi hành động tối ưu với số bước di chuyển ngắn nhất khi chi phí các bước đi bằng nhau.
- **Điểm yếu:** Tiêu tốn dung lượng bộ nhớ lớn do phải lưu trữ toàn bộ tập biên của tầng hiện tại trong quá trình duyệt. Tốc độ xử lý suy giảm nghiêm trọng và dễ gây tràn bộ nhớ khi đối mặt với các bản đồ Sokoban kích thước lớn có không gian trạng thái phát sinh theo cấp số nhân.

b. Tìm kiếm theo chiều sâu (DFS - Depth-First Search)

- **Mô tả:** Thuật toán ưu tiên phát triển nhánh tìm kiếm xuống mức sâu nhất có thể trước khi thực hiện quay lui nhờ cấu trúc ngăn xếp (Stack - LIFO). Trong hệ thống, DFS được cải tiến bằng cách sử dụng hàm đánh giá *_dfs_evaluate* (dựa trên số lượng thùng chưa vào đích và giá trị Heuristic) nhằm chủ động sắp xếp các nút lân cận theo thứ tự ưu tiên trước khi đẩy vào ngăn xếp.
- **Điểm mạnh:** Tiết kiệm không gian bộ nhớ đáng kể so với BFS do chỉ cần lưu giữ các nút trên lộ trình hiện tại và các nút lân cận chưa duyệt của chúng.
- **Điểm yếu:** Không đảm bảo tính tối ưu. Không đảm bảo tính đầy đủ trong không gian trạng thái vô hạn và dễ bị kẹt vào các nhánh cụt hoặc vòng lặp vô tận nếu không cấu hình chặt chẽ cơ chế kiểm tra trạng thái trùng.

c. Tìm kiếm chi phí đồng nhất (UCS - Uniform Cost Search)

- **Mô tả:** Thuật toán mở rộng các trạng thái dựa trên chi phí đường đi tích lũy $g(n)$ tăng dần từ trạng thái khởi đầu, vận hành thông qua hàng đợi ưu tiên (Priority Queue). Mỗi bước đẩy thùng hoặc chuyển dịch vị trí của nhân vật sẽ làm tăng độ dài chi phí thực tế.
- **Điểm mạnh:** Đảm bảo tìm thấy lời giải tối ưu tuyệt đối về mặt chi phí đường đi ngay cả khi các hành động đẩy thùng và di chuyển được thiết lập các trọng số chi phí khác nhau.
- **Điểm yếu:** Duyệt và mở rộng nút theo mọi hướng lan tỏa từ tâm mà không có thông tin định hướng đến vị trí mục tiêu, dẫn đến việc phải xử lý rất nhiều trạng thái dư thừa, làm giảm hiệu năng thực thi trên bản đồ rộng.

2. Nhóm thuật toán tìm kiếm có thông tin (Informed Search)

a. Tìm kiếm tham lam (Greedy Best-First Search)

- **Mô tả:** Thuật toán sử dụng hàm Heuristic $h(n)$ để ước lượng khoảng cách từ vị trí hiện tại tới đích, luôn ưu tiên mở rộng các nút có giá trị ước lượng này nhỏ nhất thông qua hàng đợi ưu tiên. Hàm *greedy_Heuristic* trong hệ thống được bổ sung thêm thành phần hình phạt khi thùng nằm sát tường và một lượng nhiễu ngẫu nhiên để tối ưu hóa việc chọn nhánh.

- **Điểm mạnh:** Tốc độ tìm kiếm và xử lý nhanh, tập trung mạnh về phía mục tiêu.
- **Điểm yếu:** Không đảm bảo tính tối ưu của lời giải. Dễ bị đánh lừa bởi các bẫy hình học hoặc rơi vào các trạng thái bế tắc do chỉ chọn trạng thái có vẻ gần đích nhất về mặt không gian địa hình.

b. Thuật toán A*

- **Mô tả:** Thuật toán tìm kiếm tối ưu kết hợp đồng thời giữa chi phí thực tế đã đi $g(n)$ và chi phí ước lượng đến đích $h(n)$ dựa theo hàm đánh giá tổng hợp $f(n) = g(n) + h(n)$. Quá trình chọn nút và quản lý tập biên được thực hiện qua hàng đợi ưu tiên dựa trên giá trị $f(n)$.
- **Điểm mạnh:** Đảm bảo tìm thấy lời giải tối ưu nhất nếu hàm Heuristic được thiết kế là chấp nhận được (không ước lượng vượt quá chi phí thực tế). Đạt sự cân bằng cao giữa hiệu năng tìm kiếm và độ chính xác của chuỗi hành động kết quả.
- **Điểm yếu:** Hiệu quả vận hành phụ thuộc hoàn toàn vào độ chính xác của hàm Heuristic (tổng khoảng cách kết hợp quy hoạch động bitmask từ thùng đến đích). Chi phí duy trì bộ nhớ vẫn ở mức cao khi xử lý các bản đồ cực kỳ phức tạp do phải lưu trữ danh sách trạng thái lớn trong hash table.

c. Thuật toán Weighted A*

- **Mô tả:** Biến thể của thuật toán A* áp dụng cơ chế nhân thêm một hệ số trọng số $w > 1$ vào thành phần chi phí ước lượng Heuristic, thay đổi công thức đánh giá thành $f(n) = g(n) + w \times h(n)$. Việc gia tăng tầm ảnh hưởng của Heuristic khiến thuật toán hoạt động tiệm cận với Greedy Search nhằm đẩy nhanh tiến độ tiếp cận mục tiêu.
- **Điểm mạnh:** Tăng tốc độ tìm kiếm so với A* nguyên bản, cắt giảm số lượng trạng thái cần mở rộng trong bộ nhớ, cho phép bộ giải xử lý hiệu quả các cấu hình bản đồ trung bình và lớn.
- **Điểm yếu:** Đánh đổi tính tối ưu tuyệt đối của đường đi, chuỗi hành động trả về có thể dài hơn so với kết quả của thuật toán A* tiêu chuẩn

d. Thuật toán Beam Search

- **Mô tả:** Thuật toán tìm kiếm tối ưu hóa bộ nhớ bằng cách giới hạn số lượng trạng thái được phép lưu trữ và mở rộng tại mỗi tầng ở một mức cố định (Beam Width K). Nếu số lượng nút vượt ngưỡng K , hệ thống dùng hàm *heapq.nsmallest* để giữ lại K nút có điểm Heuristic tốt nhất và giải phóng toàn bộ các nút còn lại. Khi không tìm ra giải pháp, hệ thống sẽ tự động nhân độ rộng chùm lên 2.5 lần để tiếp tục dò tìm.
- **Điểm mạnh:** Kiểm soát và giới hạn cực kỳ chặt chẽ dung lượng bộ nhớ tiêu thụ, loại bỏ hoàn toàn rủi ro tràn bộ nhớ hệ thống ngay cả trên những bản đồ có không gian trạng thái khổng lồ.
- **Điểm yếu:** Không đảm bảo tính đầy đủ và tính tối ưu của lời giải. Thuật toán có thể bỏ sót lộ trình dẫn đến trạng thái đích nếu lộ trình đó vô tình bị loại khỏi top K nút tốt nhất tại một thời điểm cắt tỉa.

V. CÁC KỸ THUẬT TỐI ƯU CỐT LỖI

1. Hashing Trạng thái với Zobrist Keys (64-bit)

Sử dụng giá trị ngẫu nhiên 64-bit sinh trước cho từng ô trên lưới (dành riêng cho player và box). Hàm băm của một trạng thái được tính bằng phép toán XOR toàn bộ các khóa liên quan. Mã băm số học này giải quyết triệt để vấn đề va chạm băm (Cache Collision) và gia tăng tốc độ tra cứu HashSet khi hàng triệu nhánh tìm kiếm được sinh ra.

2. Gom nhóm di chuyển (Macro-moves)

Loại bỏ việc coi bước đi người chơi là các hành động tương đương với việc đẩy thùng. Hệ thống chạy BFS nội bộ cục bộ để tìm mọi tọa độ khả thi mà người chơi có thể với tới trong cùng một trạng thái thùng. Chỉ các tọa độ đứng ngay sau lưng một kiện hàng (có thể thực hiện lệnh đẩy) mới được mở rộng. Kỹ thuật này giảm hệ số phân nhánh từ $O(4)$ xuống tiệm cận số lượng thùng hiện có trên bản đồ.

3. State-aware Heuristic (Bitmask Dynamic Programming)

Việc tính toán khoảng cách Manhattan đơn thuần gây sai số nghiêm trọng khi gặp tường kẹt.

- **Precomputed Distances (Mảng chi phí tính trước):** Khởi tạo một pha BFS ngược từ các ô đích để lập bản đồ chi phí đường đi thực tế gần nhất, với thời gian truy xuất tính bằng $O(1)$.
- **Bitmask DP:** Chạy thuật toán quy hoạch động với bitmask để tìm cách gán mảng tối ưu toàn cục giữa mảng vị trí thùng hiện tại và đích. Hàm này tính toán chi phí thấp nhất để tắt cả các thùng đều lọt hố, tránh hiện tượng Heuristic đánh giá thấp sai lệch.

4. Advanced Deadlock Detection

Thực thi các thuật toán sàng lọc trạng thái không thể giải quyết ở ba cấp độ:

- **Tính toán ô chết vô vọng:** Tính ra danh sách các tọa độ không thể có đường lui về phía đích (thường là dọc các mép tường rỗng) từ khi tải map.
- **Bế tắc khối hình học 2x2:** Thuật toán duyệt phát hiện khi tồn tại bất kỳ một khối 2x2 nào cấu thành từ thùng hoặc tường. Đây là mô hình bế tắc chết cứng trong luật đẩy thùng.
- **Freeze Deadlock:** Thuật toán lập sơ đồ phụ thuộc, kiểm tra đệ quy việc các thùng khóa chặt nhau từ hai trục dọc và ngang kẹp sát tường, ngăn việc dịch chuyển hoàn toàn. Những trạng thái dính deadlock này bị cắt bỏ khỏi cây biên.

VI. KỊCH BẢN KIỂM THỬ VÀ ĐÁNH GIÁ

Hệ thống tiến hành kiểm thử hiệu năng trên một tập dữ liệu lớn bao gồm 700 bản đồ được tích hợp sẵn. Tập dữ liệu này bao quát đa dạng các độ khó, kích thước và đặc tính không gian, đảm bảo kết quả đánh giá thuật toán mang tính khách quan.

Quá trình đánh giá được thực hiện tự động và trực quan hóa thông qua bảng dashboard Telemetry trên giao diện, đo lường thời gian thực các thông số vận hành cốt lõi:

- **SOLVED):** Ghi nhận kết quả YES (tìm thấy đường đi), NO (thất bại) hoặc CANCELED (người dùng chủ động hủy quá trình tính toán).
- **PATH:** Tổng số bước di chuyển và đẩy thùng thực tế để hoàn tất bản đồ.
- **NODES:** Số trạng thái được mở rộng, phản ánh độ bùng nổ tổ hợp của nhánh tìm kiếm.

- **TIME:** Chi phí thời gian chạy thuật toán.

Để có cơ sở so sánh đặc tính của 7 thuật toán, tập 700 bản đồ được phân rã thành 4 nhóm kịch bản thử nghiệm chính:

1. Nhóm kịch bản cơ sở (bản đồ nhỏ, cấu trúc đơn giản)

- **Đặc tính:** Lưới kích thước nhỏ, số lượng thùng ít (1-2 thùng), không gian di chuyển thoáng.
- **Mục tiêu:** Kiểm định tính đúng đắn của logic vật lý cốt lõi, xác minh khả năng sinh trạng thái lân cận hợp lệ và loại trừ lỗi di chuyển xuyên tường.

2. Nhóm kịch bản bế tắc

- **Đặc tính:** Bản đồ chứa nhiều góc chết, hành lang hẹp, dễ dàng đẩy thùng vào các vị trí không thể đảo ngược.
- **Mục tiêu:** Đo lường độ nhạy và sức mạnh cắt tỉa của module Advanced Deadlock Detection. Theo dõi sự chênh lệch về chỉ số **NODES** mở rộng trước và sau khi kích hoạt các bộ lọc Freeze Deadlock và Deadlock hình học 2x2.

3. Nhóm kịch bản phân nhánh cao (bản đồ trung bình, nhiều cụm thùng)

- **Đặc tính:** Số lượng thùng và ô đích lớn (từ 4-8 thùng), tồn tại nhiều ngã rẽ và các phương án điều phối thứ tự đẩy thùng khác nhau.
- **Mục tiêu:** Đánh giá năng lực của các hàm Heuristic. So sánh tốc độ hội tụ về đích của A* và Greedy Search khi được trang bị kỹ thuật Bitmask DP so với sự suy giảm hiệu năng bùng nổ trạng thái của nhóm thuật toán duyệt mù (BFS, UCS). Đánh giá sự đánh đổi giữa chỉ số **TIME** và **PATH**.

VII. ĐỊNH HƯỚNG PHÁT TRIỂN DỰ ÁN

Nhằm nâng cấp năng lực tính toán và hoàn thiện dự án, nhóm đề xuất 4 hướng phát triển chuyên sâu:

- **Cải tiến thuật toán & Heuristic:** Tích hợp cơ sở dữ liệu mẫu (Pattern Databases) để tối ưu hàm ước lượng $h(n)$ và áp dụng các kỹ thuật cắt tỉa bậc cao (Tunnel/PI-Corral Pruning) nhằm triệt tiêu các nhánh duyệt vô vọng.
- **Ứng dụng Machine Learning:** Tích hợp học tăng cường (Reinforcement Learning - DQN/PPO) để tác tử tự học chính sách giải quyết, hoặc huấn luyện mạng nơ-ron (CNN) dự đoán chi phí Heuristic từ ma trận điểm ảnh.
- **Tối ưu hóa nền tảng tính toán:** Tái cấu trúc phân lớp lõi AI (solver_utils.py, algorithms.py) bằng ngôn ngữ cấp thấp (C++/Rust) và liên kết qua Native Bindings nhằm tháo gỡ nút thắt cổ chai GIL của Python, bứt phá tốc độ mở rộng nút.
- **Phát triển công cụ hỗ trợ:** Xây dựng bộ công cụ thiết kế bản đồ trực quan (Visual Level Editor) và chế độ gỡ lỗi (AI Debugger) với bản đồ nhiệt để theo dõi, phân tích từng bước đi của thuật toán trực tiếp trên giao diện.